

Graph Neural Networks

Credit:

- Jure Leskovec
- Chenghan Jiang

A data-driven perspective on graph problems

Problem instances may follow a distribution.

Graphs from the same application may share recurring patterns in their structure, features, or weights.

Past instances can therefore contain useful information for solving new instances.

Learning from past instances

Machine learning can capture recurring patterns.

Training: learn from examples of graph instances and their solutions or outcomes.

Use: apply the learned model to help solve new instances from a similar setting.

The challenge: learning from graph data

The connections are part of the input.

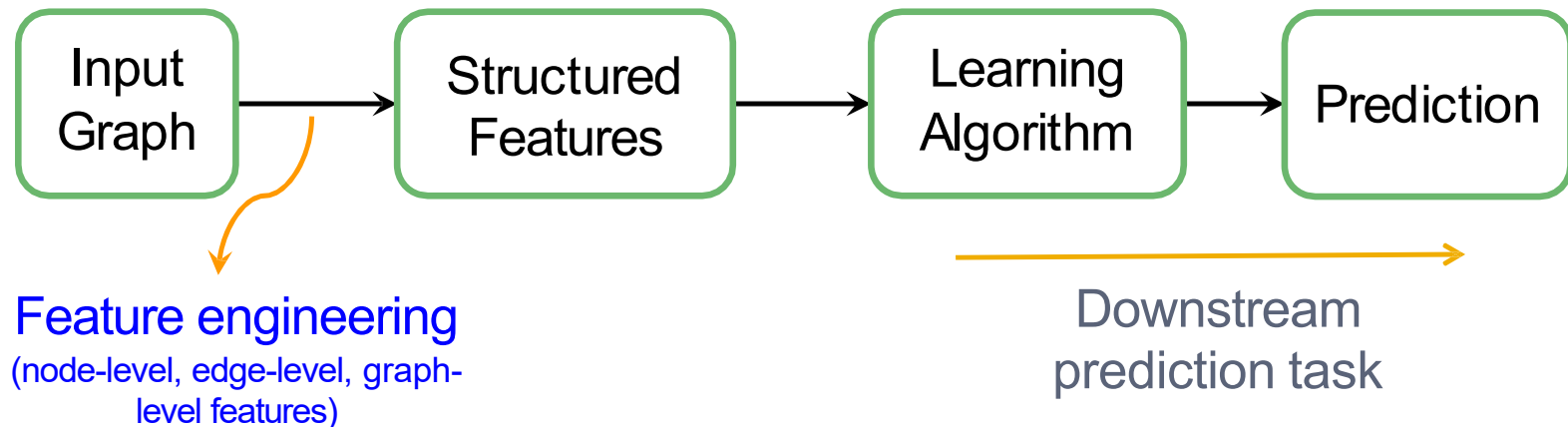
Graphs can have different numbers of nodes and edges.

Nodes have no natural ordering. Renaming them should not change the meaning of the graph.

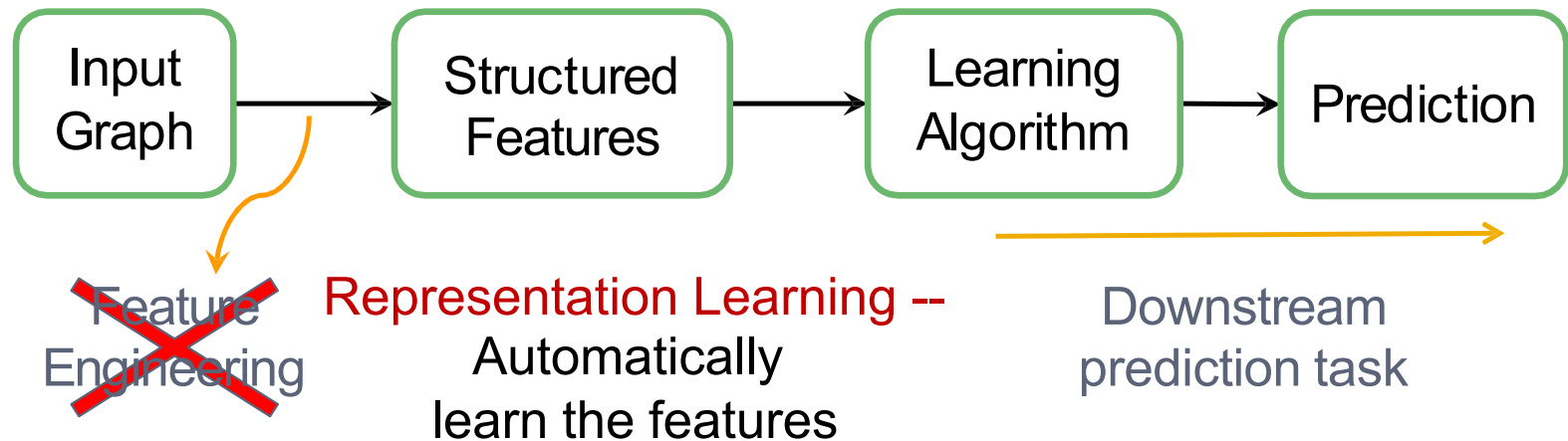
A useful model must capture both attributes and relationships.

Traditional ML Pipeline

Given an input graph, extract node, link and graph-level features, learn a model (SVM, neural network, etc.) that maps features to labels.

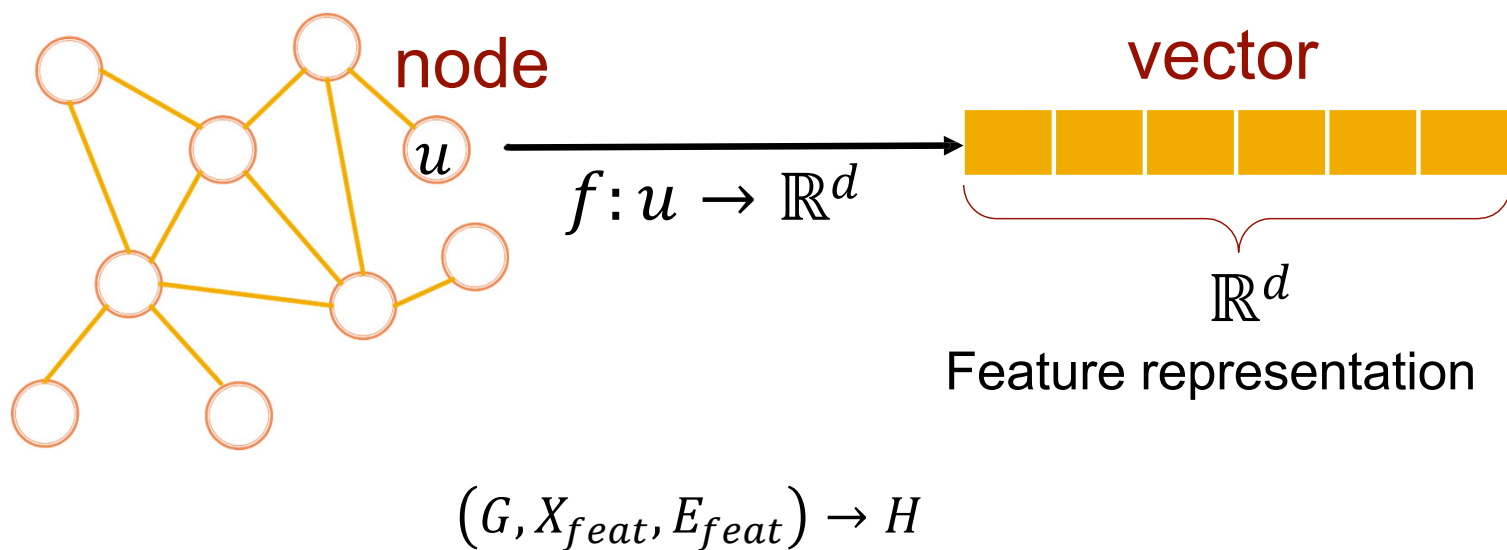


Graph Representation Learning



Graph Representation Learning

Goal: Efficient task-independent feature learning for machine learning with graphs



Graph neural networks

Neural networks designed to learn from graphs.

Input: graph structure, with optional node and edge features.

Core idea: nodes combine their own information with information from their neighbors.

Output: learned representations for prediction or for guiding a graph algorithm.

The Basics: Graph Neural Networks

Based on material from:

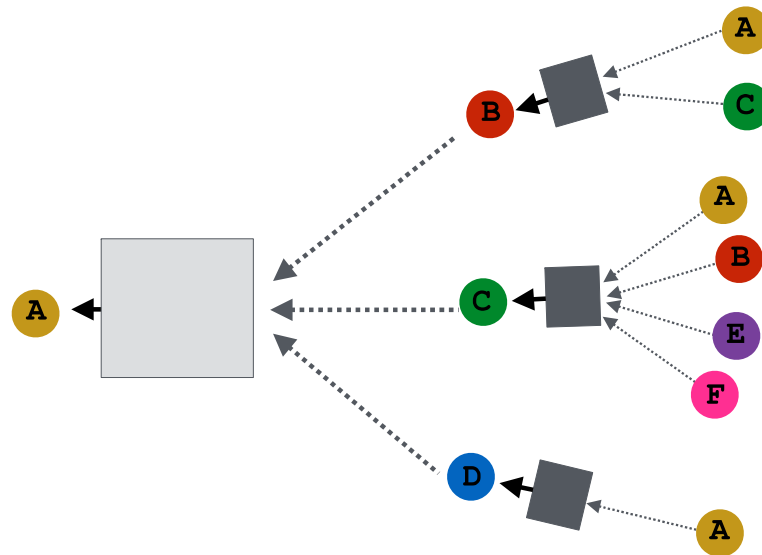
- Hamilton et al. 2017. [Representation Learning on Graphs: Methods and Applications](#). *IEEE Data Engineering Bulletin on Graph Systems*.
- Scarselli et al. 2005. [The Graph Neural Network Model](#). *IEEE Transactions on Neural Networks*.

Setup

- Assume we have a graph G :
 - V is the vertex set.
 - A is the adjacency matrix (assume binary).
 - $X \in \mathbb{R}^{m \times |V|}$ is a matrix of node features.
 - Categorical attributes, text, image data
 - E.g., profile information in a social network.
 - Node degrees, clustering coefficients, etc.

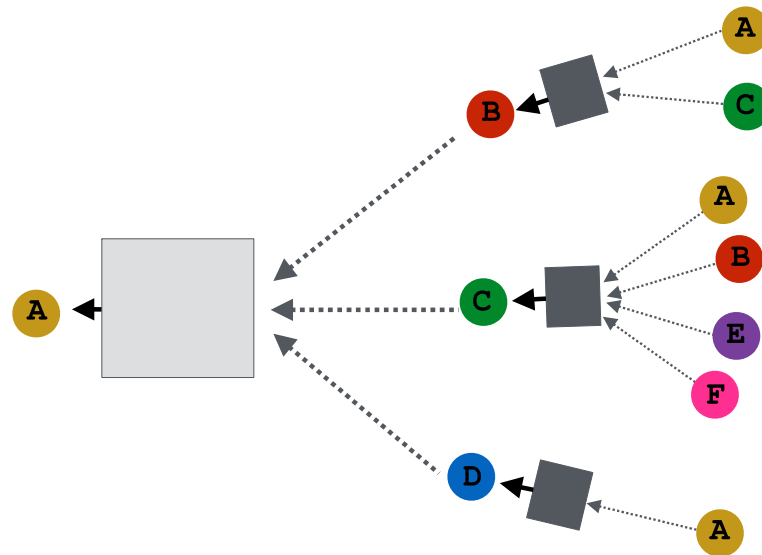
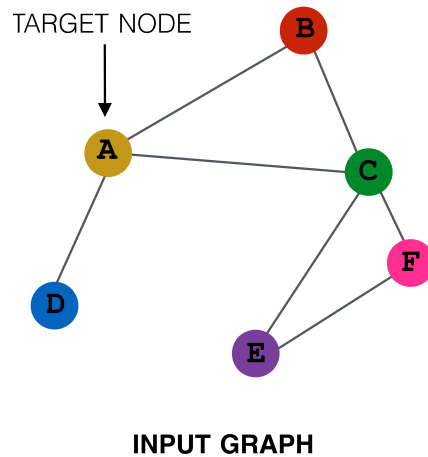
Neighborhood Aggregation

- **Key idea:** Generate node embeddings based on local neighborhoods.



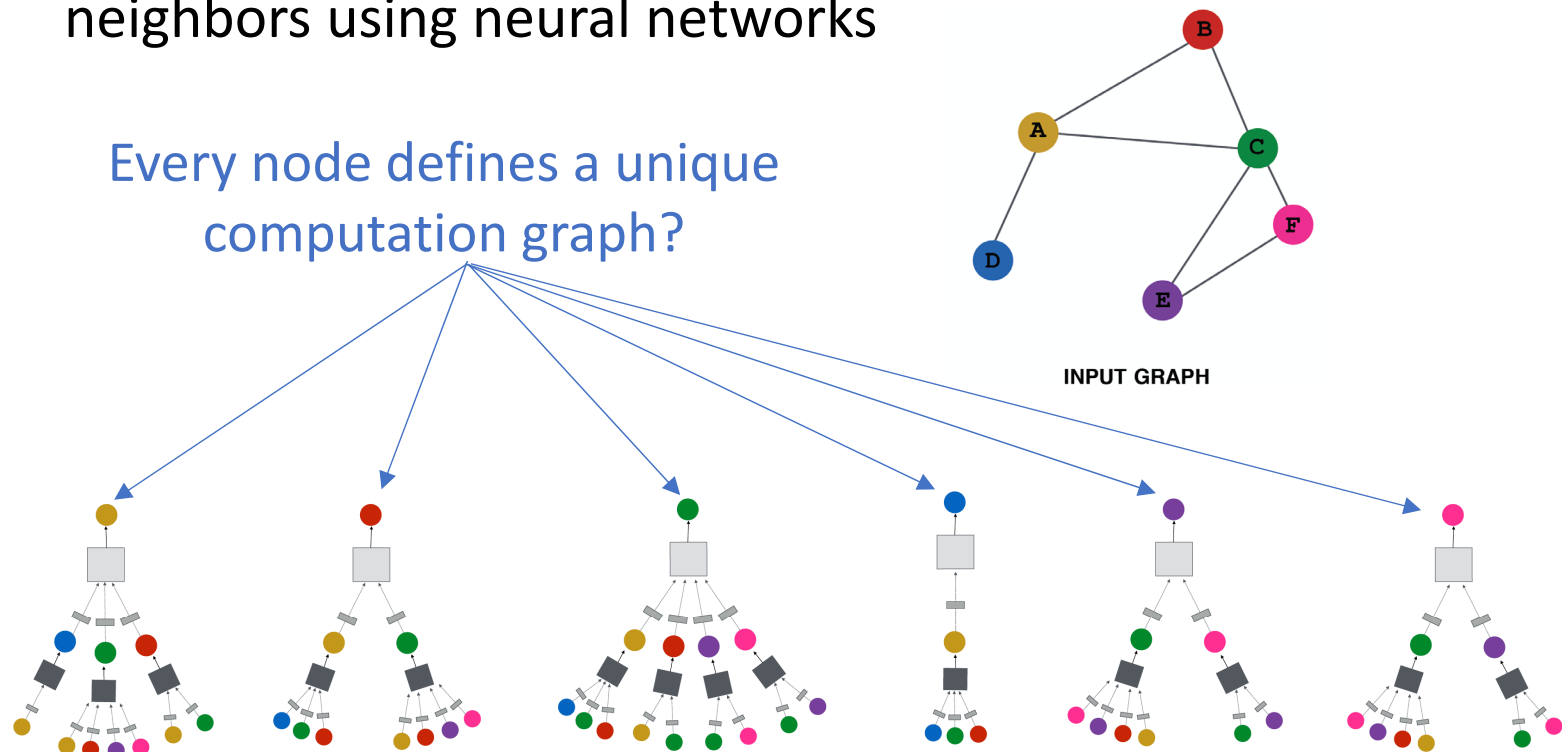
Neighborhood Aggregation

- **Intuition:** Nodes aggregate information from their neighbors using neural networks



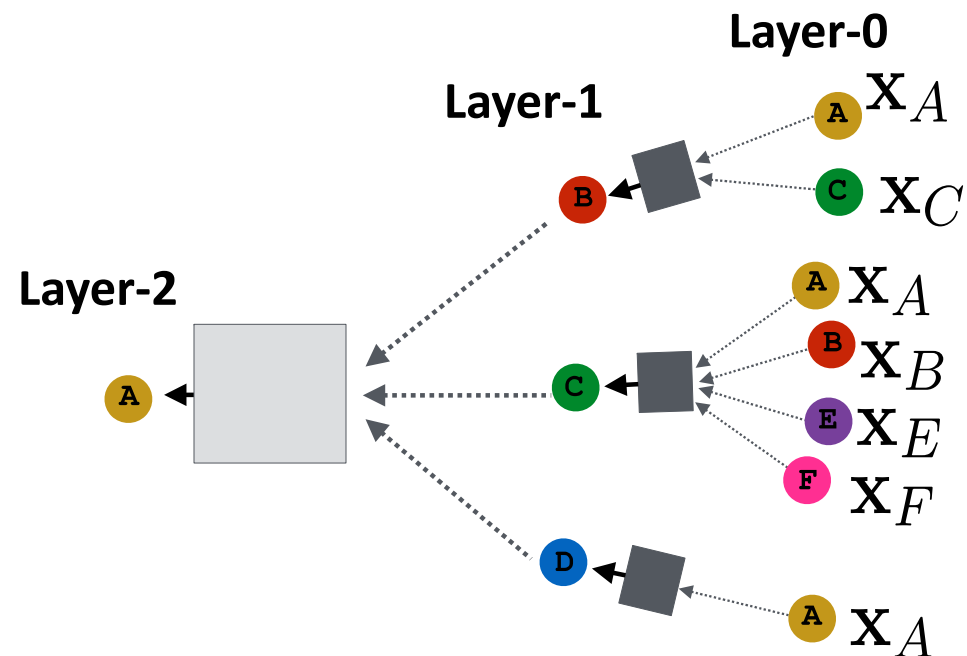
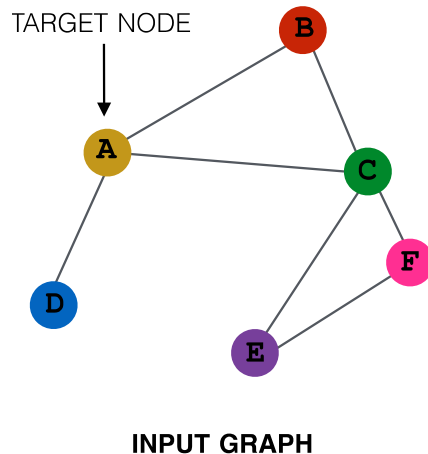
Neighborhood Aggregation

- **Intuition:** Nodes aggregate information from their neighbors using neural networks



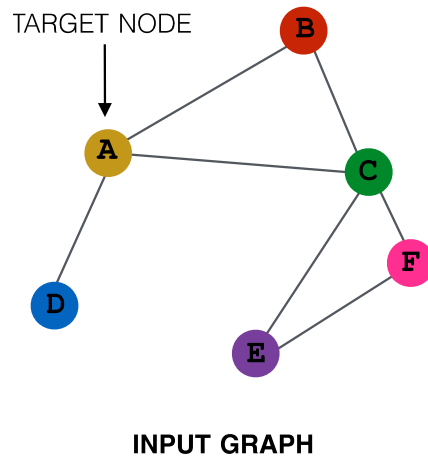
Neighborhood Aggregation

- Nodes have **embeddings** of features at each layer.
- Model can be arbitrary depth.
- “layer-0” embedding of node u is its input feature, i.e. x_u .

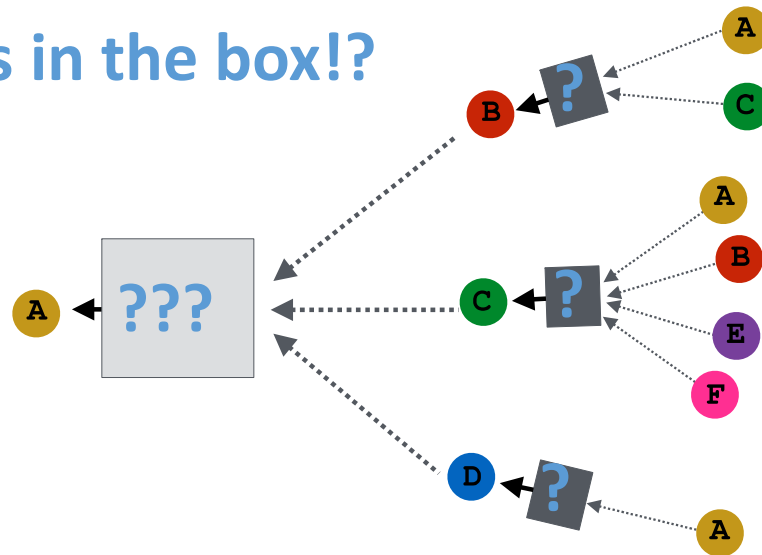


Neighborhood Aggregation

- Key distinctions are in how different approaches aggregate information across the layers.

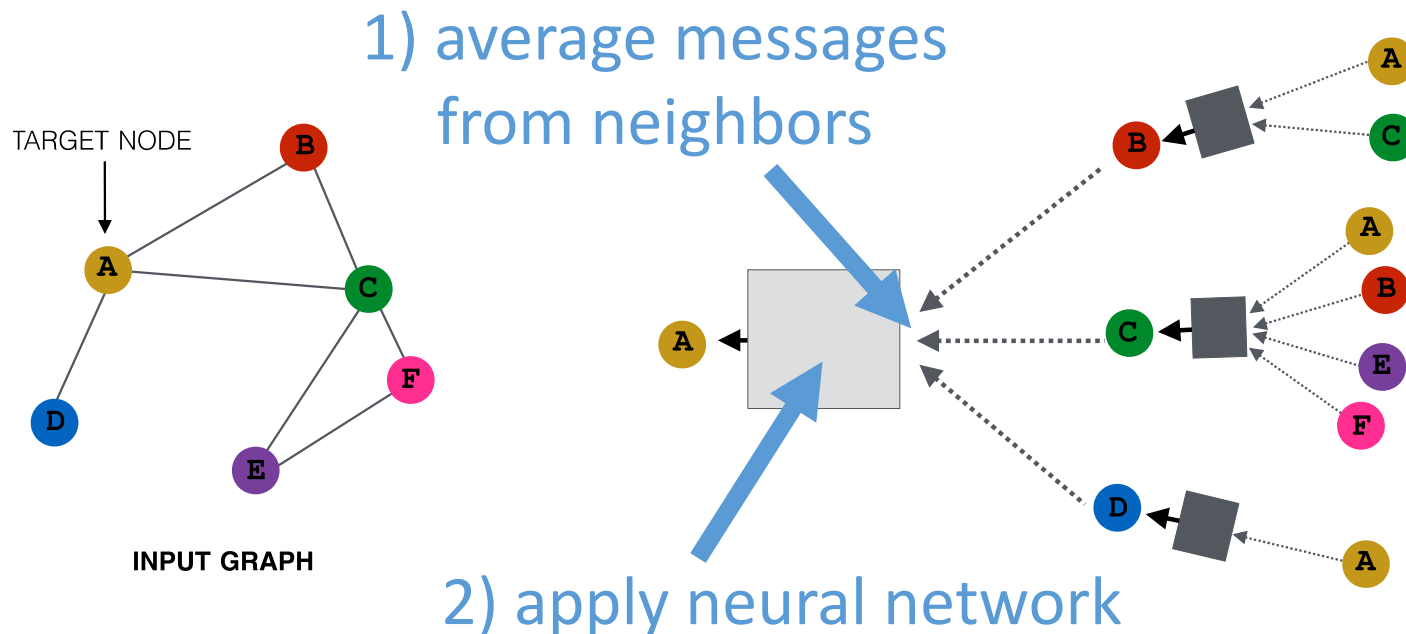


what's in the box!?



Neighborhood Aggregation

- **Basic approach:** Average neighbor information and apply a neural network.



The Math

- **Basic approach:** Average neighbor messages and apply a neural network.

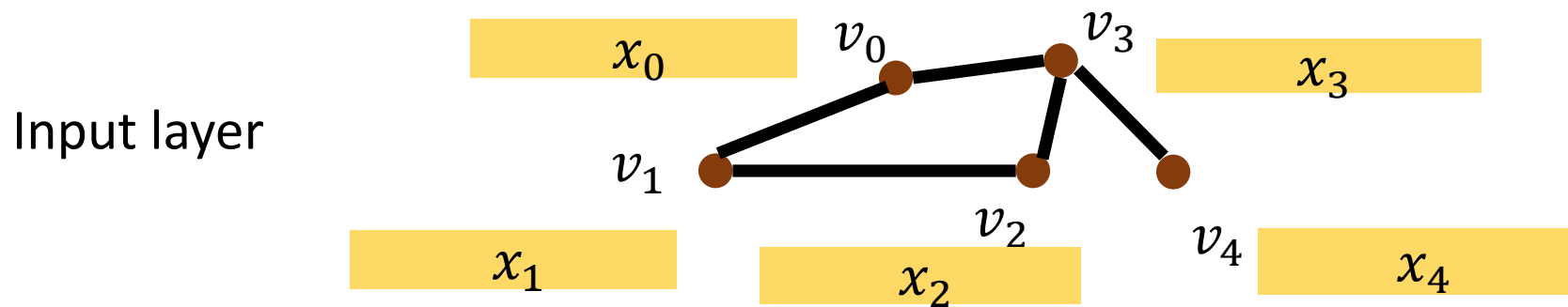
The diagram illustrates the mathematical formulation of a graph neural network layer. The equation is:

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right), \quad \forall k > 0$$

Annotations and components:

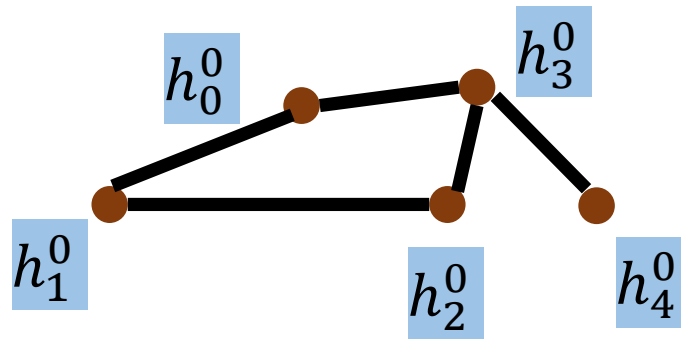
- Initial "layer 0" embeddings are equal to node features:** Points to the initial embedding $\mathbf{h}_v^0 = \mathbf{x}_v$ (yellow box).
- previous layer embedding of v :** Points to the previous layer embedding $\mathbf{h}_v^{k-1}$ (green box).
- average of neighbor's previous layer embeddings:** Points to the summation term $\sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$ (orange box).
- non-linearity (e.g., ReLU or tanh):** Points to the activation function σ (blue box).
- kth layer embedding of v :** Points to the final embedding $\mathbf{h}_v^k$ (gray box).

Example

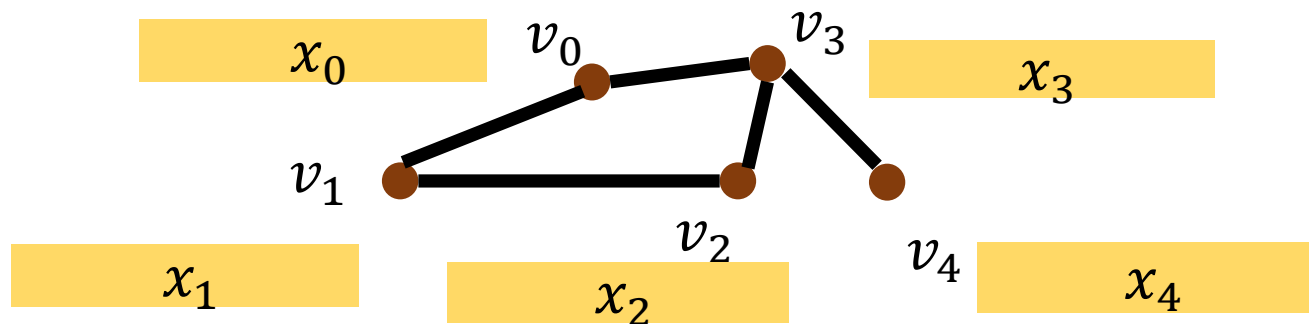


Example

Hidden layer 0:

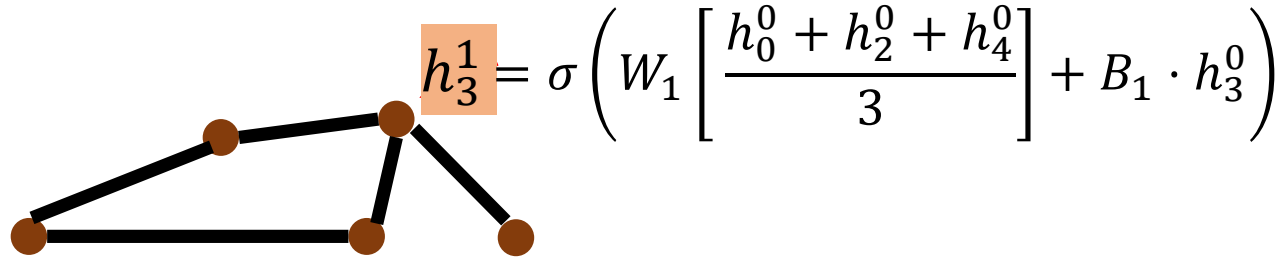


Input layer

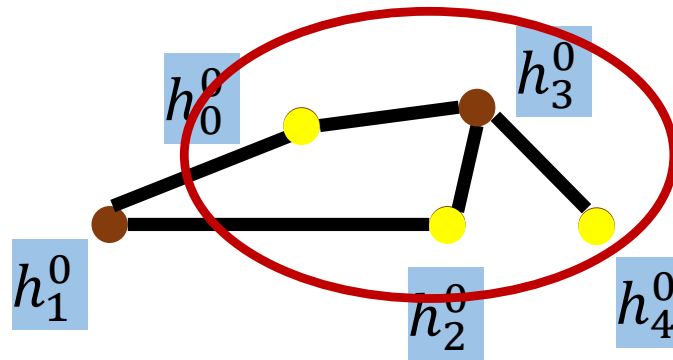


Example

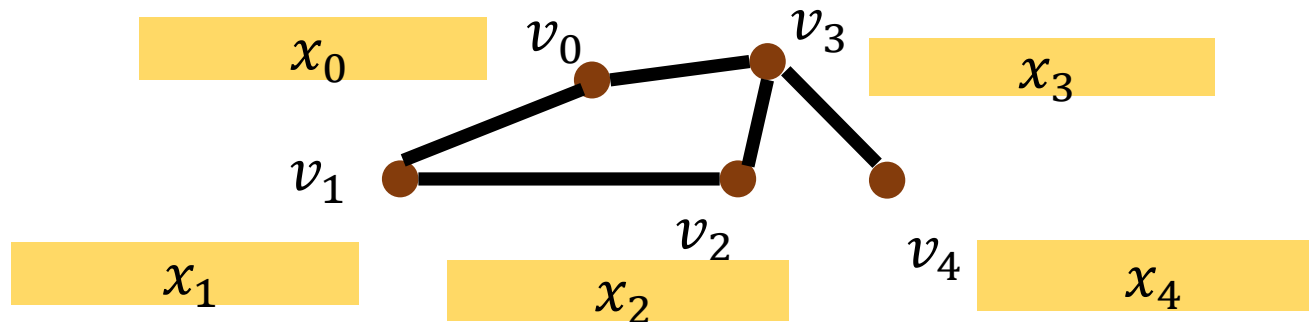
Hidden layer 1:



Hidden layer 0:

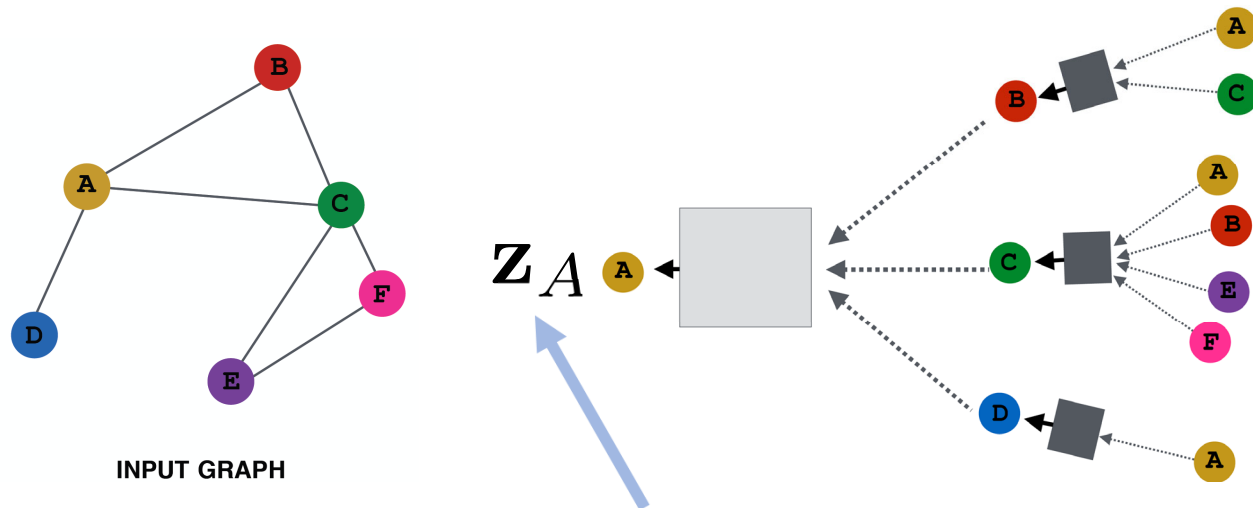


Input layer



Training the Model

- How do we train the model to generate “high-quality” embeddings?



Need to define a loss function on the embeddings, $L(z_u)$!

Training the Model

trainable matrices
(i.e., what we learn)

$$\mathbf{h}_v^0 = \mathbf{x}_v$$
$$\mathbf{h}_v^k = \sigma \left(\boxed{\mathbf{W}_k} \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \boxed{\mathbf{B}_k} \mathbf{h}_v^{k-1} \right), \quad \forall k \in \{1, \dots, K\}$$

$\boxed{\mathbf{z}_v = \mathbf{h}_v^K}$

- After K-layers of neighborhood aggregation, we get output embeddings for each node.
- **We can feed these embeddings into any loss function** and run stochastic gradient descent to train the **aggregation parameters**.

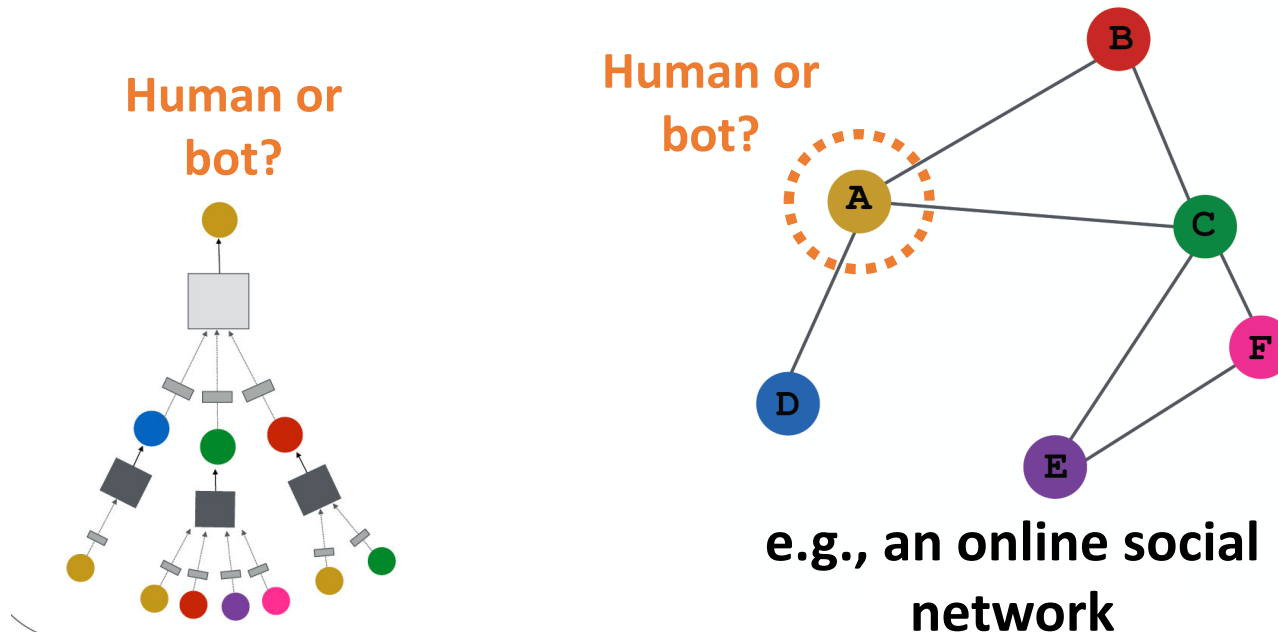
Training the Model

- Train in an **unsupervised manner** using only the graph structure.
- Unsupervised loss function can be, e.g.,
 - Random walks (node2vec, DeepWalk)
 - Matrix factorization
 - i.e., train the model so that “similar” nodes have similar embeddings.

$$\text{ENC}(v) = \mathbf{z}_v \qquad \text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$$

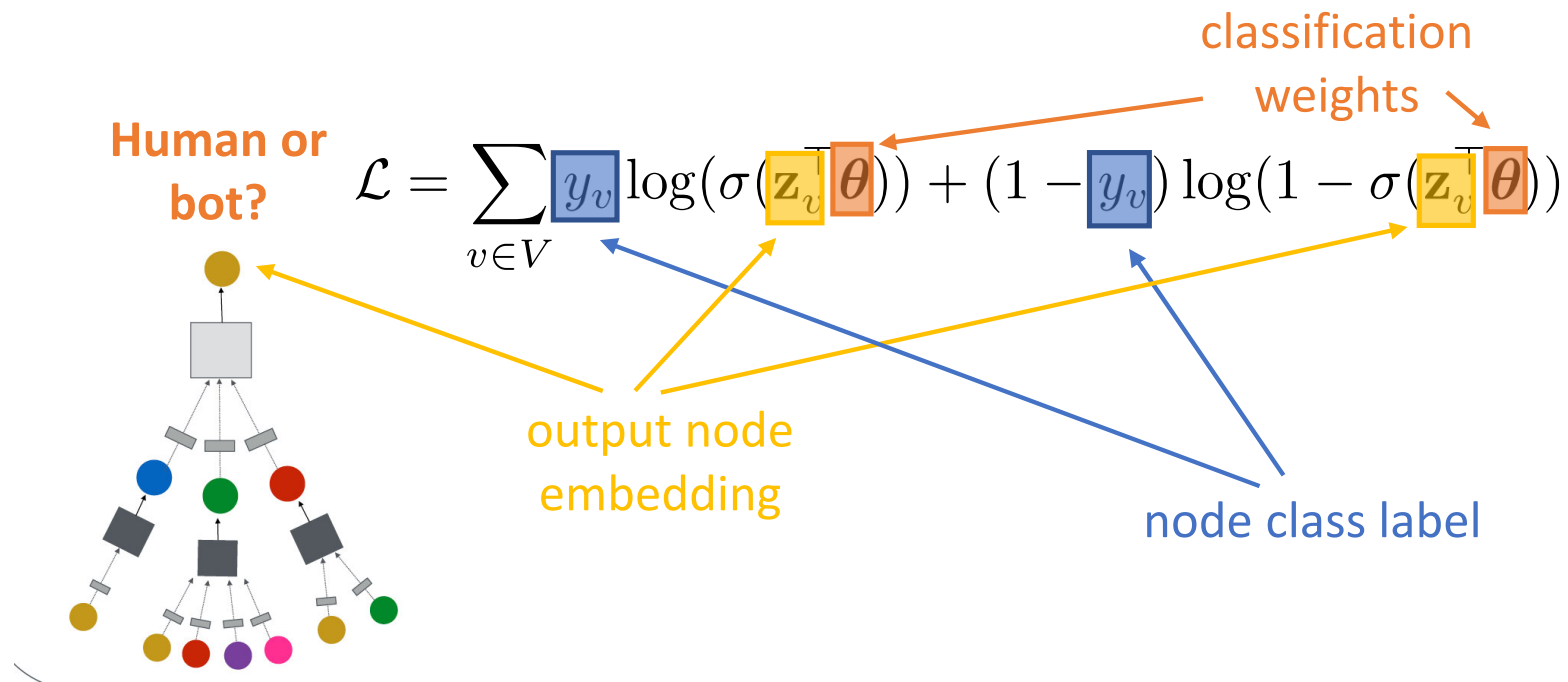
Training the Model

- **Alternative:** Directly train the model for a supervised task (e.g., node classification):



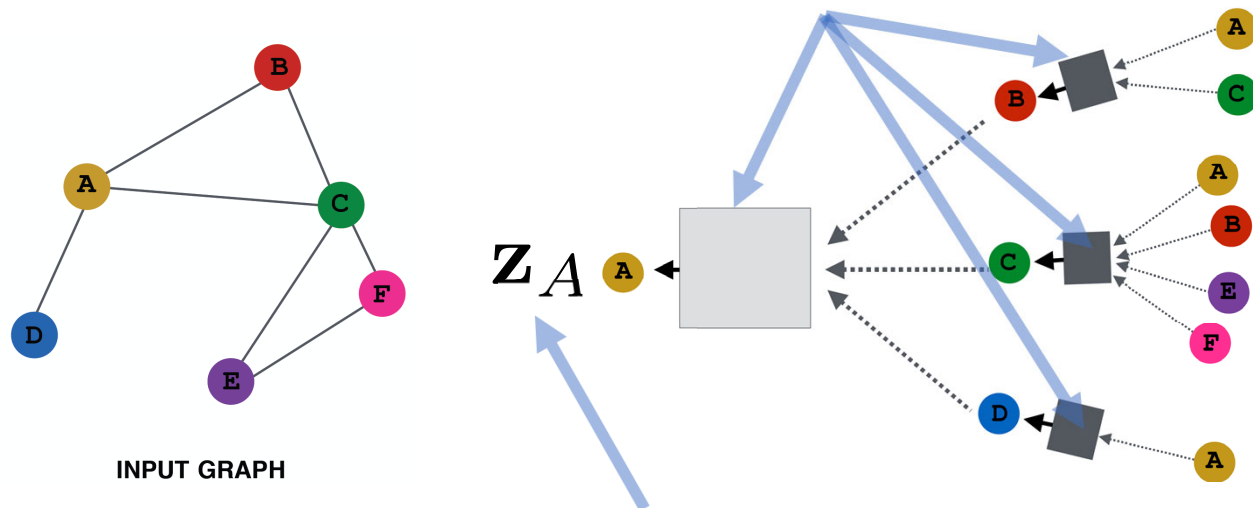
Training the Model

- **Alternative:** Directly train the model for a supervised task (e.g., node classification):



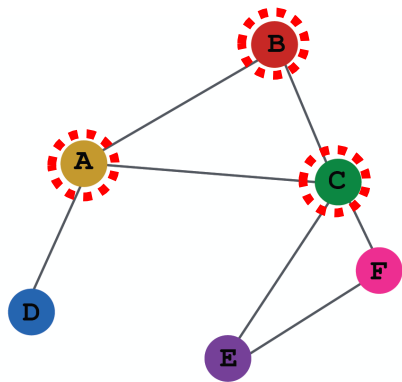
Overview of Model Design

1) Define a neighborhood aggregation function.



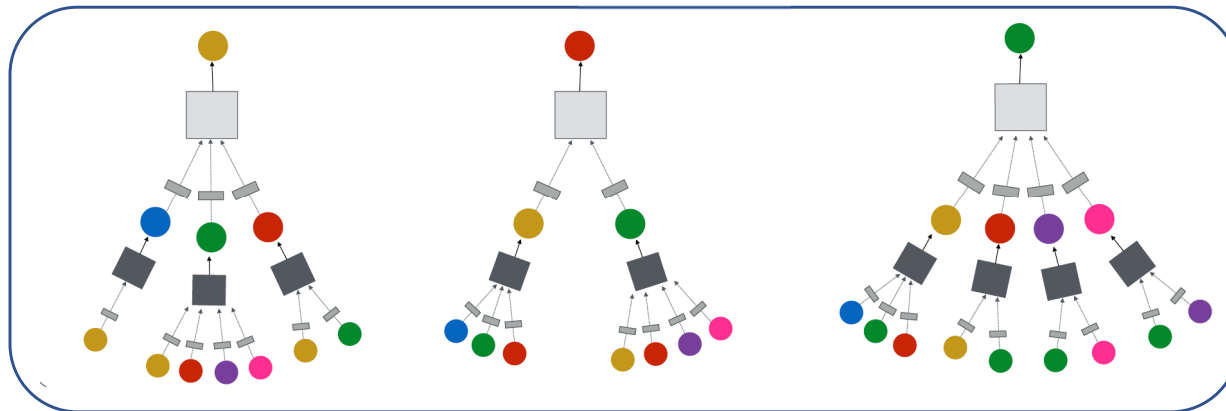
2) Define a loss function on the embeddings, $L(z_u)$

Overview of Model Design

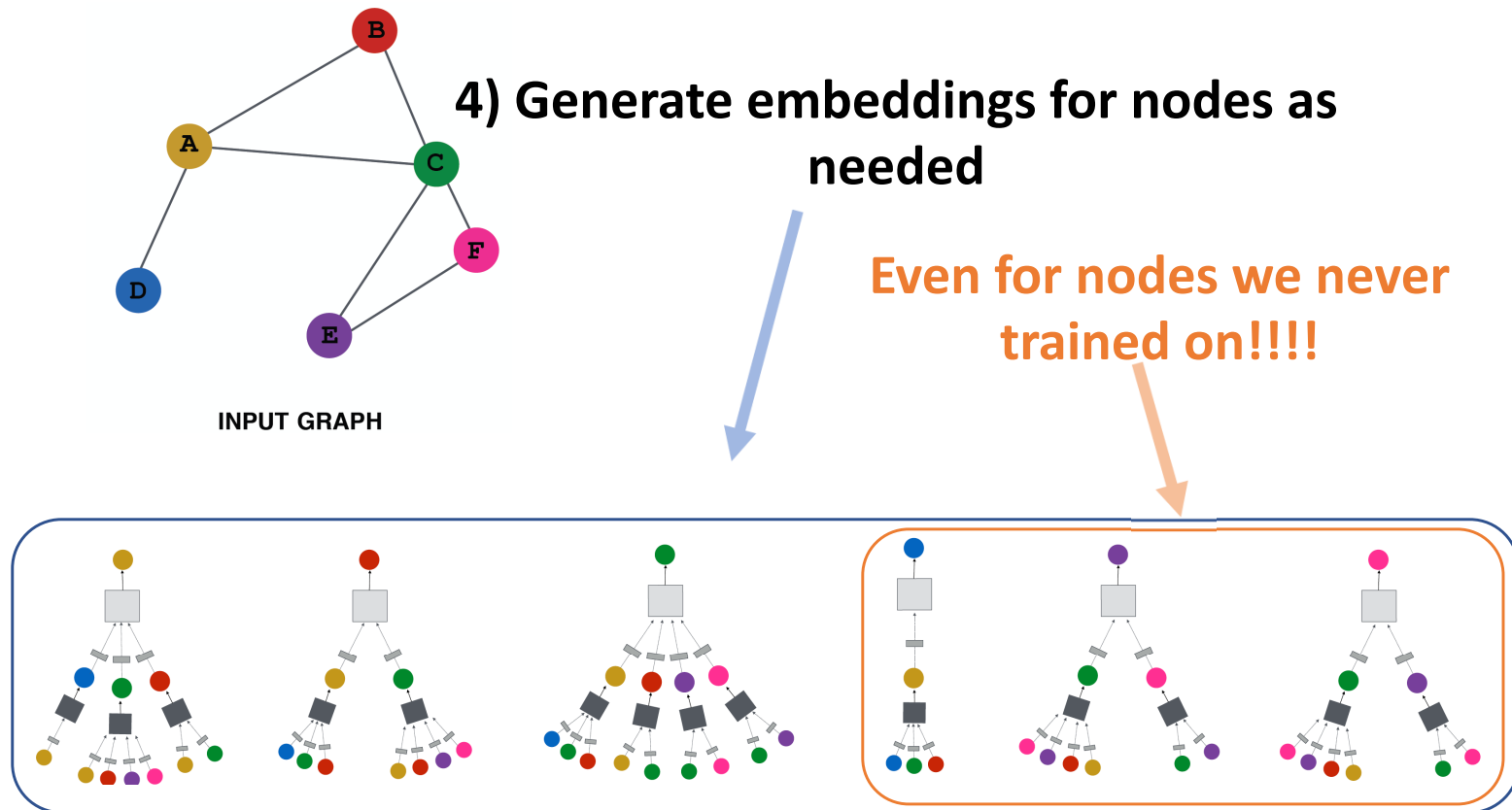


INPUT GRAPH

3) Train on a set of nodes, i.e., a batch of compute graphs

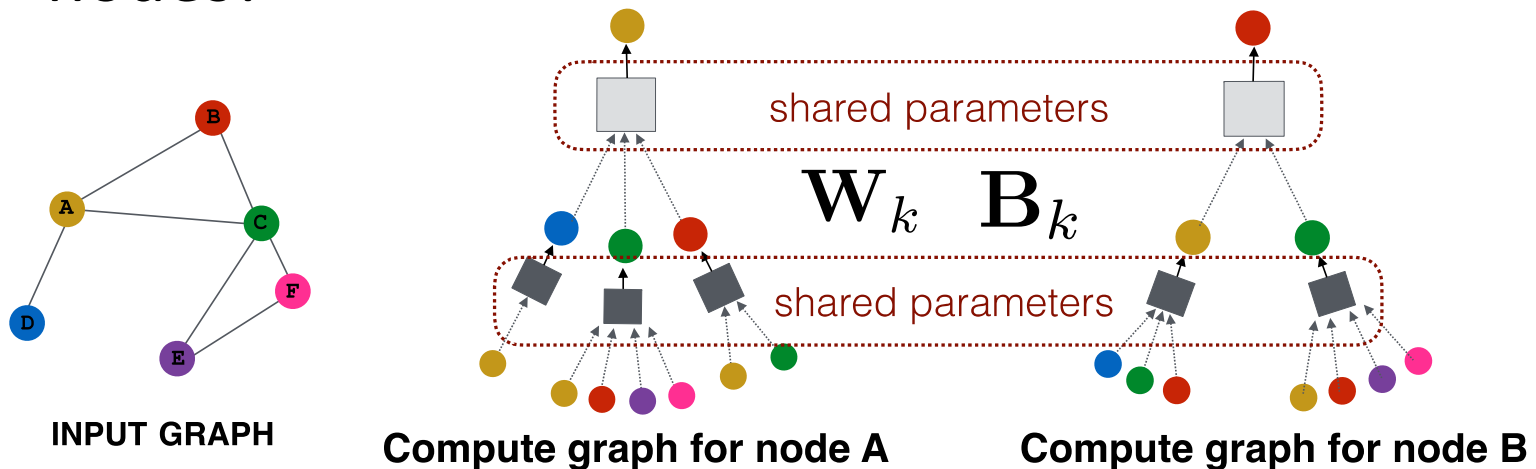


Overview of Model

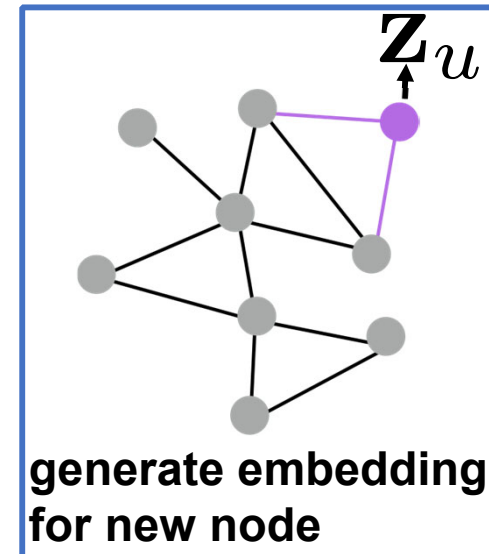
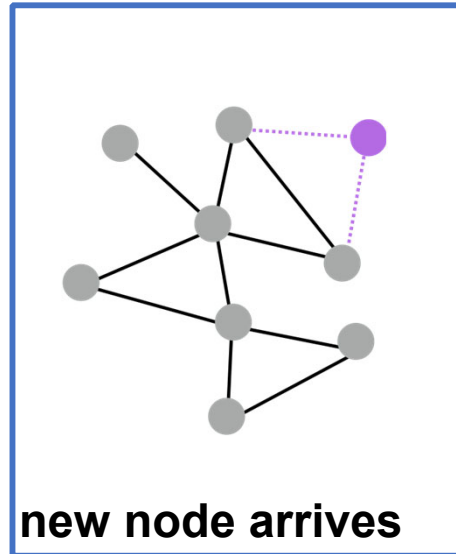
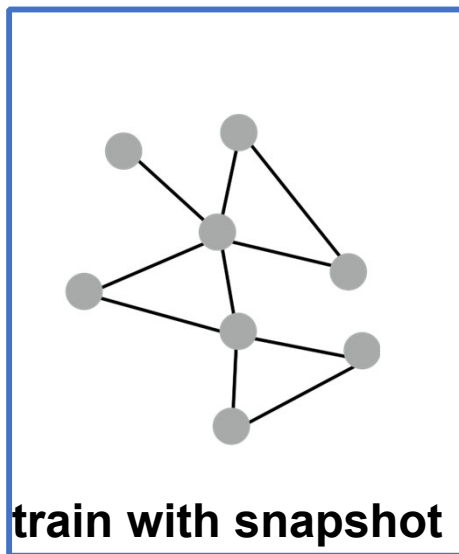


Inductive Capability

- The same aggregation parameters are shared for all nodes.
- The number of model parameters is sublinear in $|V|$ and we can generalize to unseen nodes!



Inductive Capability

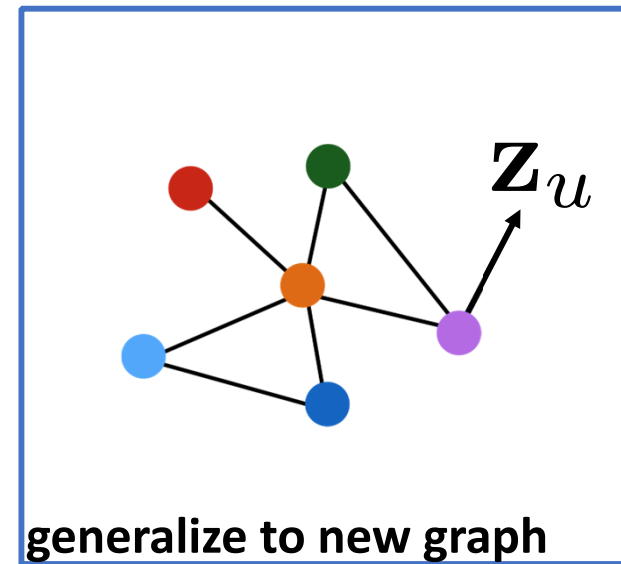
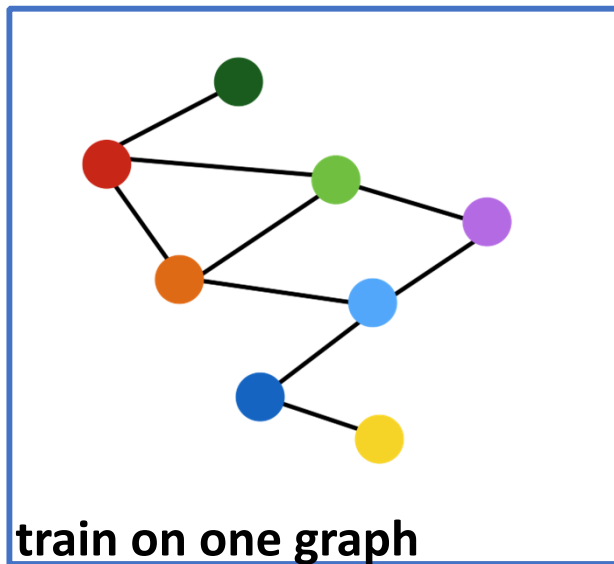


Many application settings constantly encounter previously unseen nodes.

e.g., Reddit, YouTube, Google Scholar,

Need to generate new embeddings “on the fly”

Inductive Capability



Inductive node embedding → generalize to entirely unseen graphs

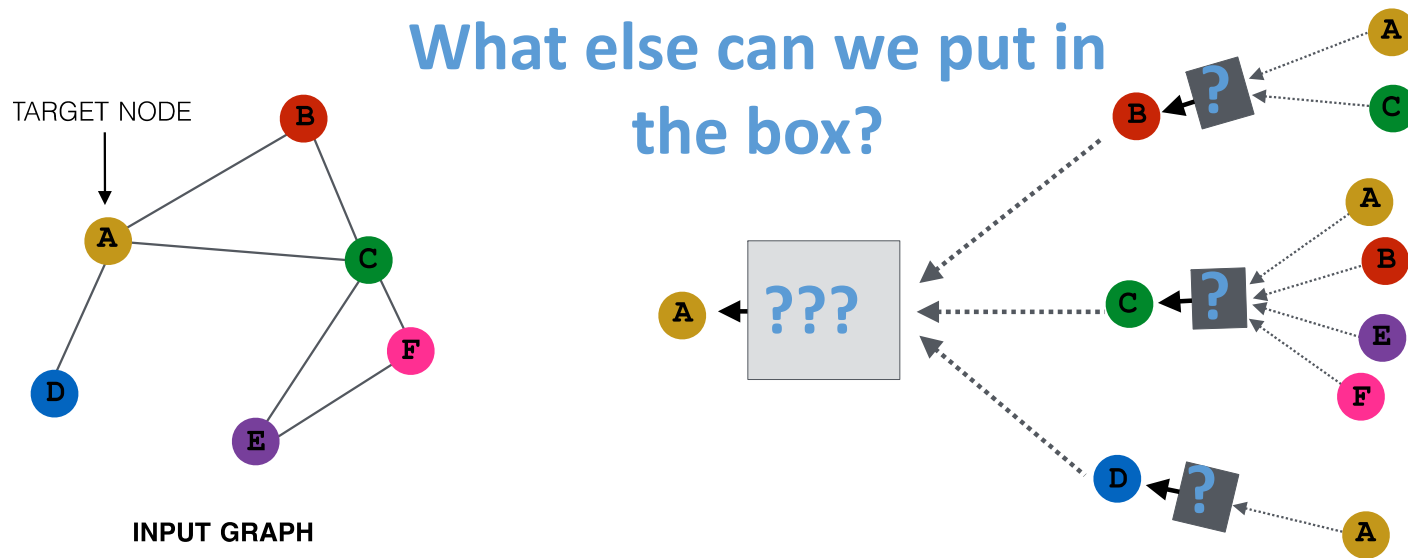
e.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Quick Recap

- **Recap:** Generate node embeddings by aggregating neighborhood information.
 - Allows for parameter sharing in the encoder.
 - Allows for inductive learning.
- We saw a **basic variant of this idea**... now we will cover some state of the art variants from the literature.

Neighborhood Aggregation

- Key distinctions are in how different approaches aggregate messages

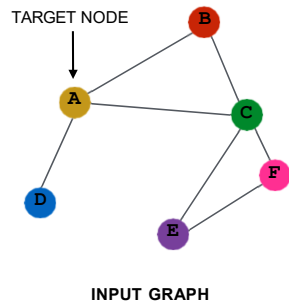


Outline for this Section

1. The Basics ✓
2. Graph Convolutional Networks
3. GraphSAGE
4. Gated Graph Neural Networks
5. Subgraph Embeddings

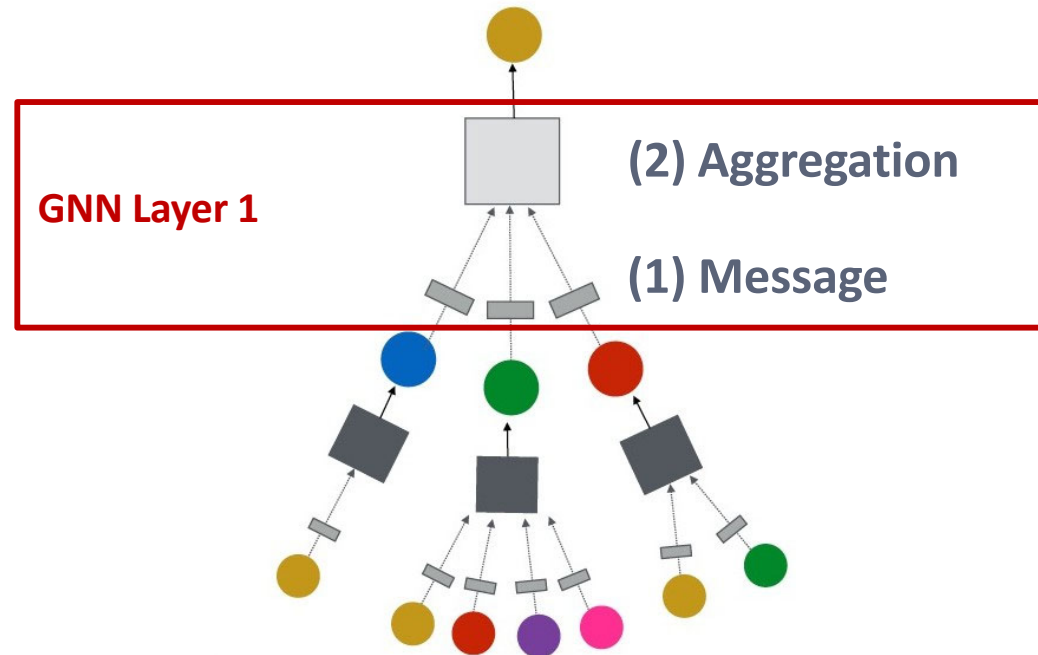


A GNN Layer



GNN Layer = Message + Aggregation

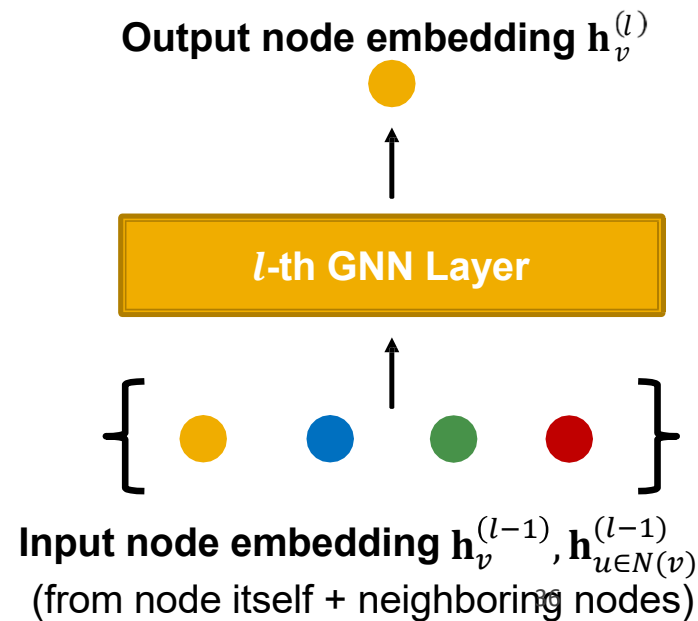
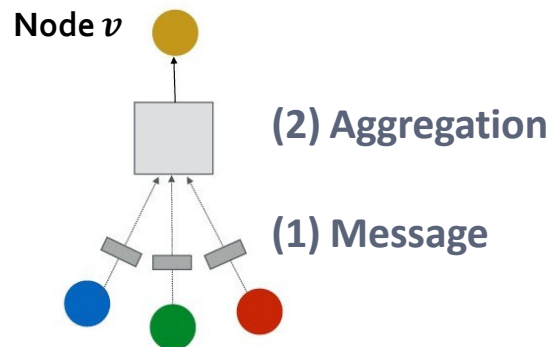
- Different instantiations under this perspective
- GCN, GraphSAGE, GAT, ...



A Single GNN Layer

- **Idea of a GNN Layer:**

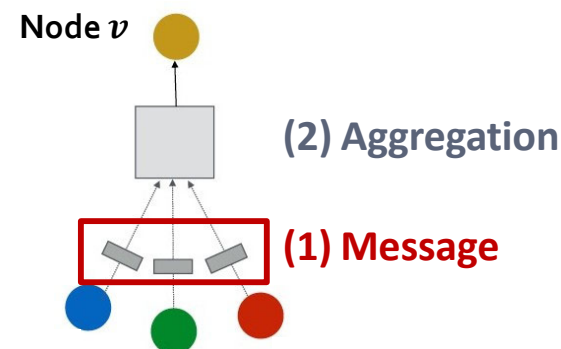
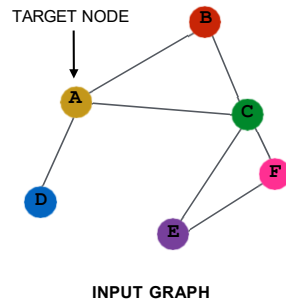
- Compress a set of vectors into a single vector
- **Two-step process:**
 - (1) Message
 - (2) Aggregation



Message Computation

■ (1) Message computation

- **Message function:** $\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right)$
 - **Intuition:** Each node will create a message, which will be sent to other nodes later
 - **Example:** A Linear layer $\mathbf{m}_u^{(l)} = \mathbf{W}^l \mathbf{h}_u^{(l-1)}$
 - Multiply node features with weight matrix $\mathbf{W}^{(l)}$



Message Aggregation

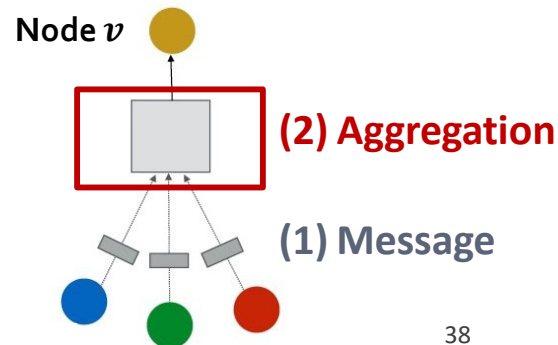
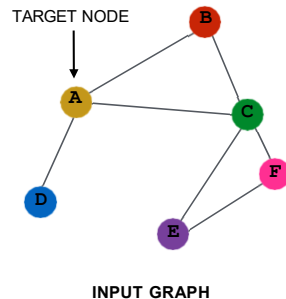
■ (2) Aggregation

- **Intuition:** Each node will aggregate the messages from node v 's neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$$

- **Example:** Sum($\cdot$), Mean($\cdot$) or Max($\cdot$) aggregator

- $\mathbf{h}_v^{(l)} = \text{Sum} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\} \right)$



Message Aggregation: Issue

- **Issue:** Information from node v itself **could get lost**
- **Solution:** Include $\mathbf{h}_v^{(l-1)}$ when computing $\mathbf{h}_v^{(l)}$
 - **(1) Message:** compute message from node v itself
 - Usually, a **different message computation** will be performed
 - **(2) Aggregation:** After aggregating from neighbors, we can **aggregate the message from node v itself**
 - Via **concatenation** or **summation**

A Single GNN Layer

- **Putting things together:**

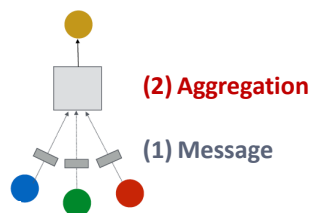
- **(1) Message:** each node computes a message

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

- **(2) Aggregation:** aggregate messages from neighbors

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

- **Nonlinearity (activation):** Adds expressiveness
 - Often written as $\sigma(\cdot)$: $\text{ReLU}(\cdot)$, $\text{Sigmoid}(\cdot)$, ...
 - Can be added to **message** or **aggregation**



The Math

- **Basic approach:** Average neighbor messages and apply a neural network.

The diagram illustrates the mathematical formulation of a graph neural network layer. The central equation is:

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right), \quad \forall k > 0$$

Annotations and components:

- Initial "layer 0" embeddings are equal to node features:** Points to the initial embedding $\mathbf{h}_v^0 = \mathbf{x}_v$ (yellow box).
- previous layer embedding of v :** Points to the previous layer embedding $\mathbf{h}_v^{k-1}$ (green box).
- average of neighbor's previous layer embeddings:** Points to the summation term $\sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$ (orange box).
- non-linearity (e.g., ReLU or tanh):** Points to the activation function σ (blue box).
- kth layer embedding of v :** Points to the final embedding $\mathbf{h}_v^k$ (gray box).

Graph Convolutional Networks

Based on material from:

- Kipf et al., 2017. [Semisupervised Classification with Graph Convolutional Networks](#). *ICLR*.

Graph Convolutional Networks

- [Kipf et al.'s Graph Convolutional Networks \(GCNs\)](#) are a slight variation on the neighborhood aggregation idea:

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)| |N(v)|}} \right)$$

Graph Convolutional Networks

Basic Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right)$$

VS.

GCN Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)| |N(v)|}} \right)$$

same matrix for self and
neighbor embeddings

per-neighbor normalization

Graph Convolutional Networks

- Empirically, they found this configuration to give the best results.
 - More parameter sharing.
 - Down-weights high-degree neighbors.

$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)| |N(v)|}} \right)$$

use the same transformation matrix
for self and neighbor embeddings

instead of simple average,
normalization varies across
neighbors

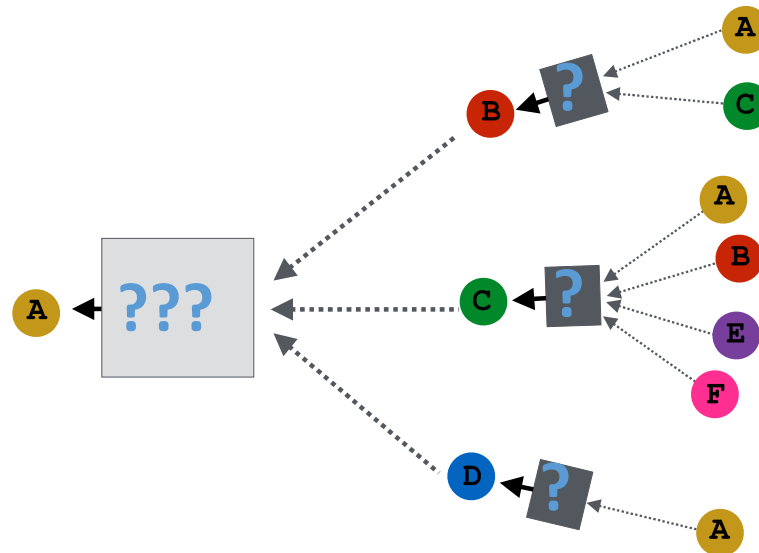
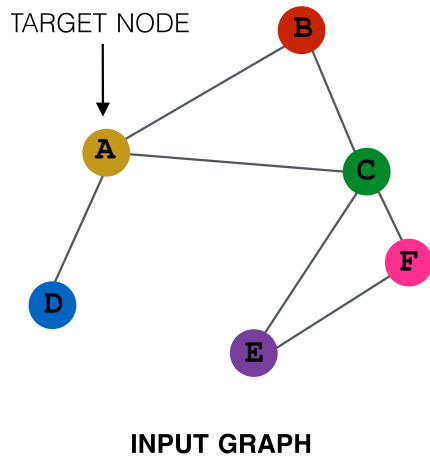
GraphSAGE

Based on material from:

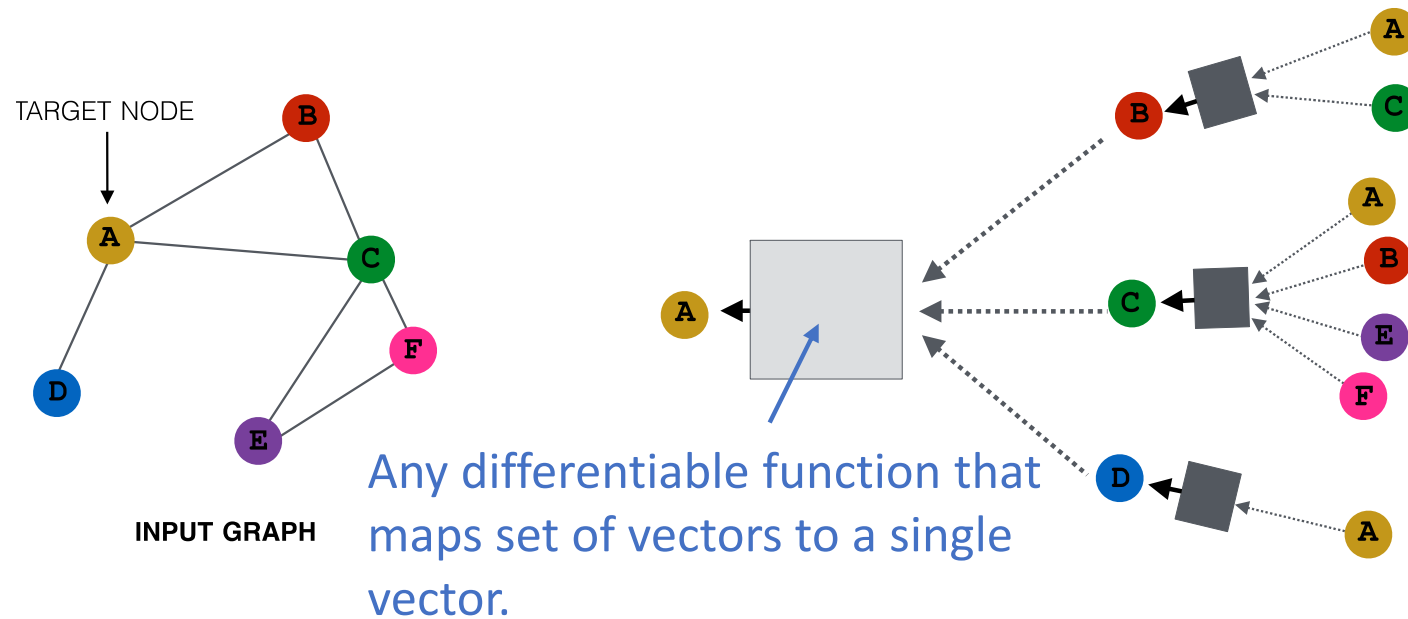
- Hamilton et al., 2017. [Inductive Representation Learning on Large Graphs](#). *NIPS*.

GraphSAGE Idea

- So far we have aggregated the neighbor messages by taking their (weighted) average, can we do better?



GraphSAGE Idea



$$\mathbf{h}_v^k = \sigma \left([\mathbf{A}_k \cdot \text{AGG}(\{\mathbf{h}_u^{k-1}, \forall u \in N(v)\}), \mathbf{B}_k \mathbf{h}_v^{k-1}] \right)$$

GraphSAGE Differences

- Simple neighborhood aggregation:

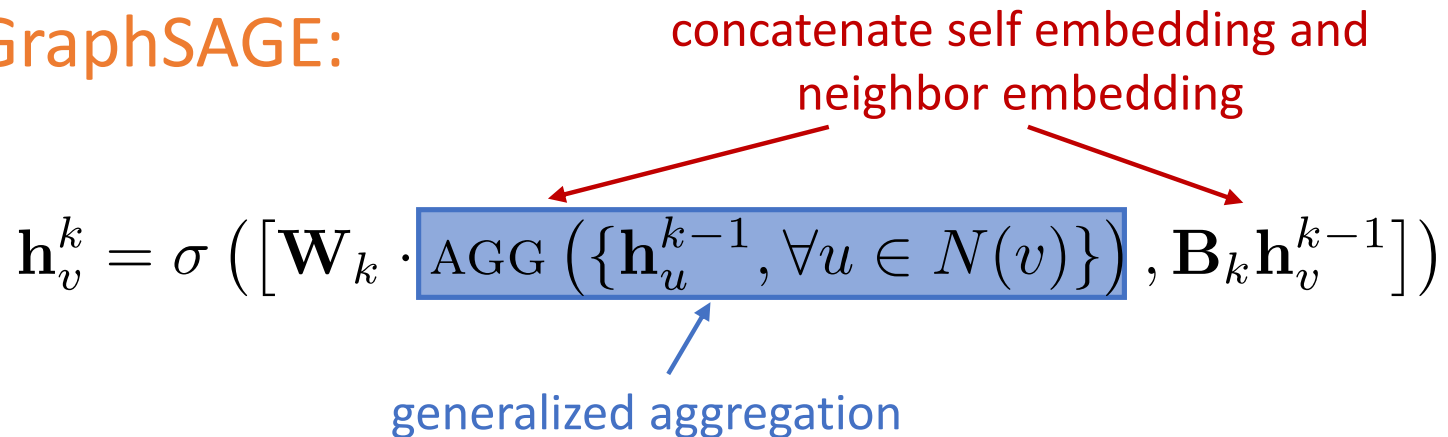
$$\mathbf{h}_v^k = \sigma \left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right)$$

- GraphSAGE:

concatenate self embedding and neighbor embedding

$$\mathbf{h}_v^k = \sigma \left(\left[\mathbf{W}_k \cdot \text{AGG} \left(\{ \mathbf{h}_u^{k-1}, \forall u \in N(v) \} \right), \mathbf{B}_k \mathbf{h}_v^{k-1} \right] \right)$$

generalized aggregation



GraphSAGE Variants

- **Mean Aggregator:**

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$

- **Pooling Aggregator**

- Transform neighbor vectors and apply symmetric vector function.

$$\text{AGG} = \gamma(\{\mathbf{Q}\mathbf{h}_u^{k-1}, \forall u \in N(v)\})$$

← element-wise mean/max

- **LSTM Aggregator:**

- Apply LSTM to random permutation of neighbors.

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))])$$

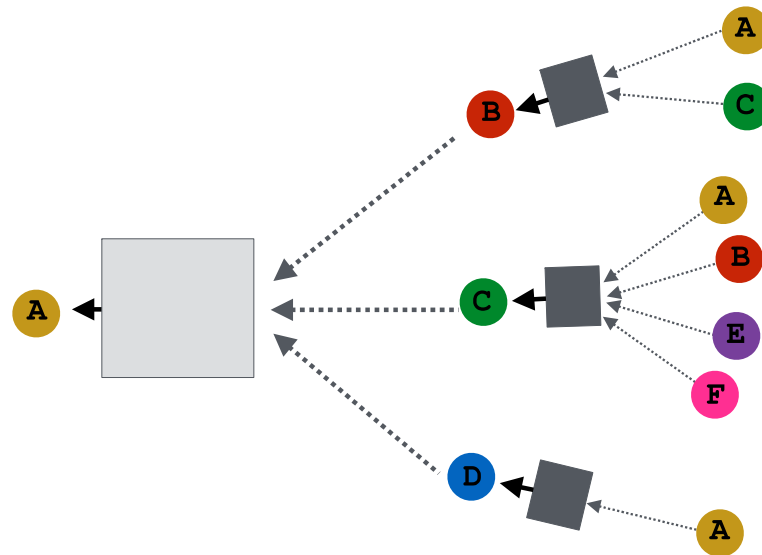
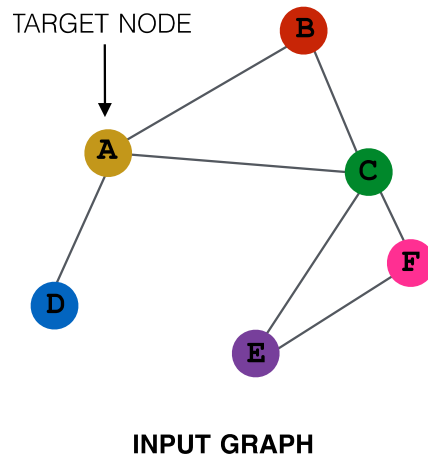
Gated Graph Neural Networks

Based on material from:

- Li et al., 2016. [Gated Graph Sequence Neural Networks](#). *ICLR*.

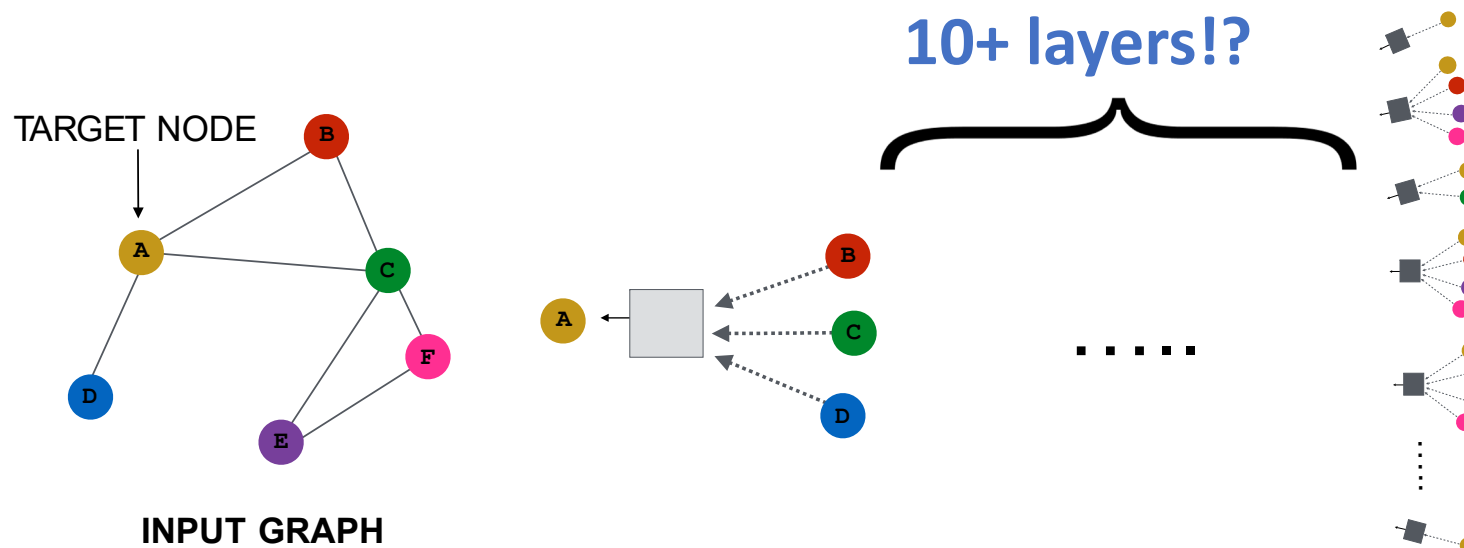
Neighborhood Aggregation

- GCNs and GraphSAGE **generally only 2-3 layers deep.**



Neighborhood Aggregation

- But what if we want to go deeper?

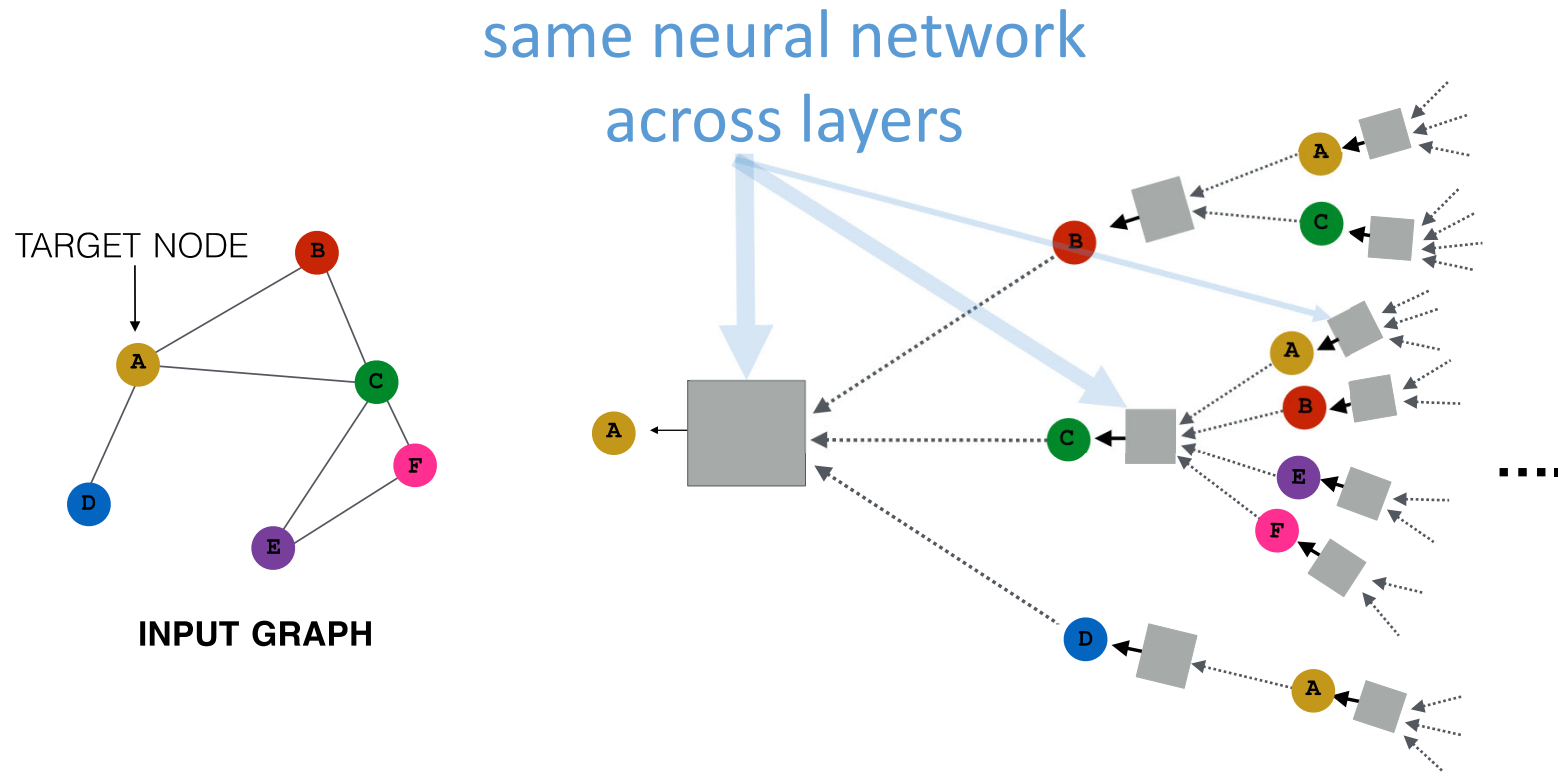


Gated Graph Neural Networks

- How can we build models with many layers of neighborhood aggregation?
- **Challenges:**
 - Overfitting from too many parameters.
 - Vanishing/exploding gradients during backpropagation.
- **Idea:** Use techniques from modern recurrent neural networks!

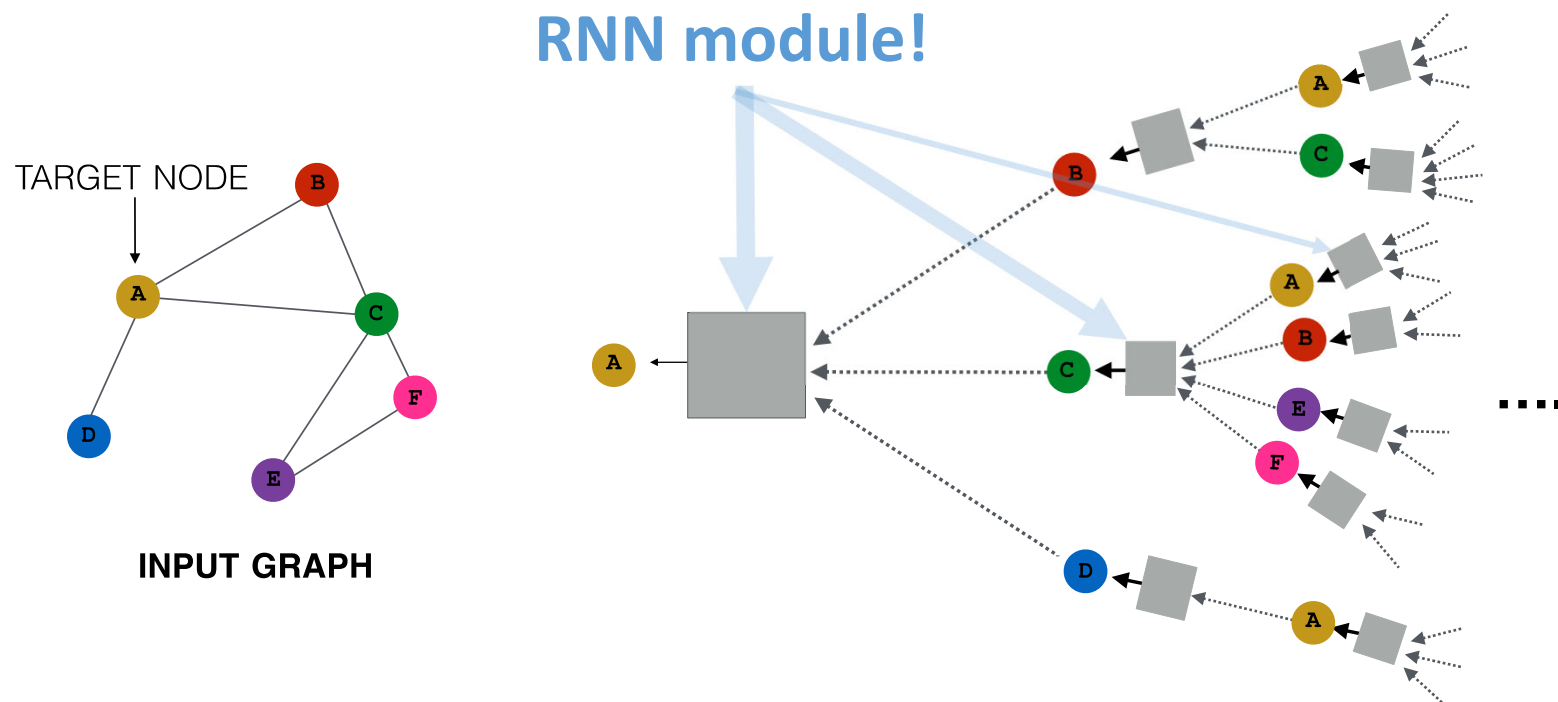
Gated Graph Neural Networks

- **Idea 1:** Parameter sharing across layers.



Gated Graph Neural Networks

- **Idea 2:** Recurrent state update.



The Math

- **Intuition:** Neighborhood aggregation with RNN state update.

1. Get “message” from neighbors at step k :

$$\mathbf{m}_v^k = \mathbf{W} \sum_{u \in N(v)} \mathbf{h}_u^{k-1}$$

← aggregation function does not depend on k

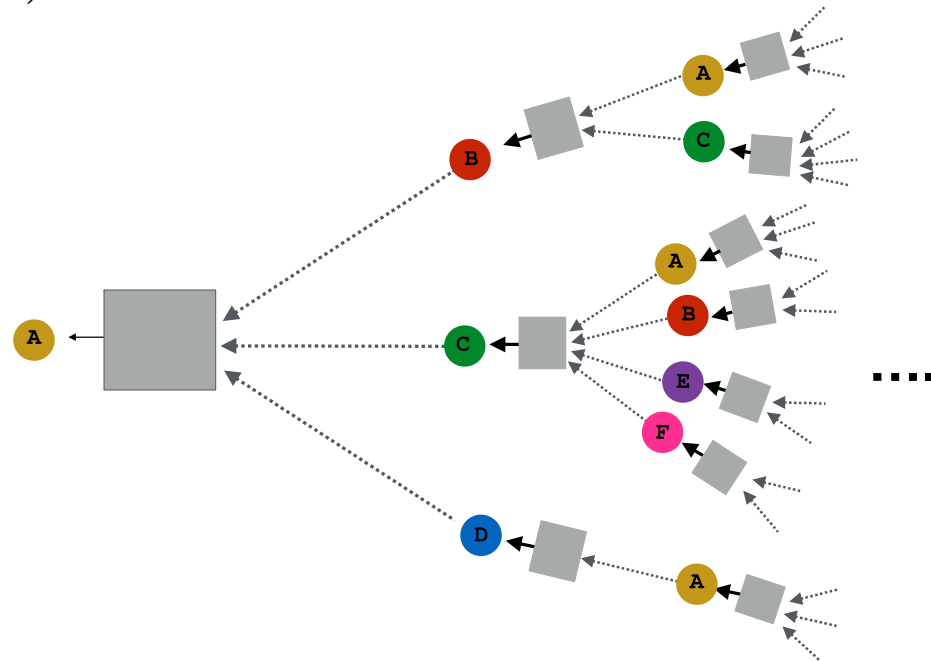
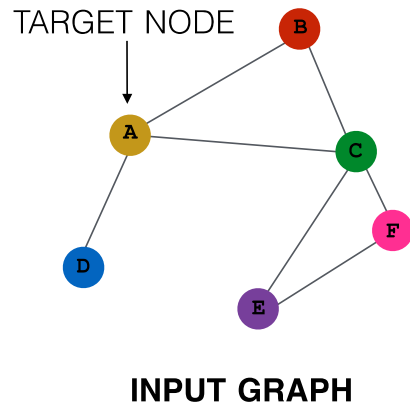
2. Update node “state” using Gated Recurrent Unit (GRU). New node state depends on the old state and the message from neighbors:

$$\mathbf{h}_v^k = \text{GRU}(\mathbf{h}_v^{k-1}, \mathbf{m}_v^k)$$

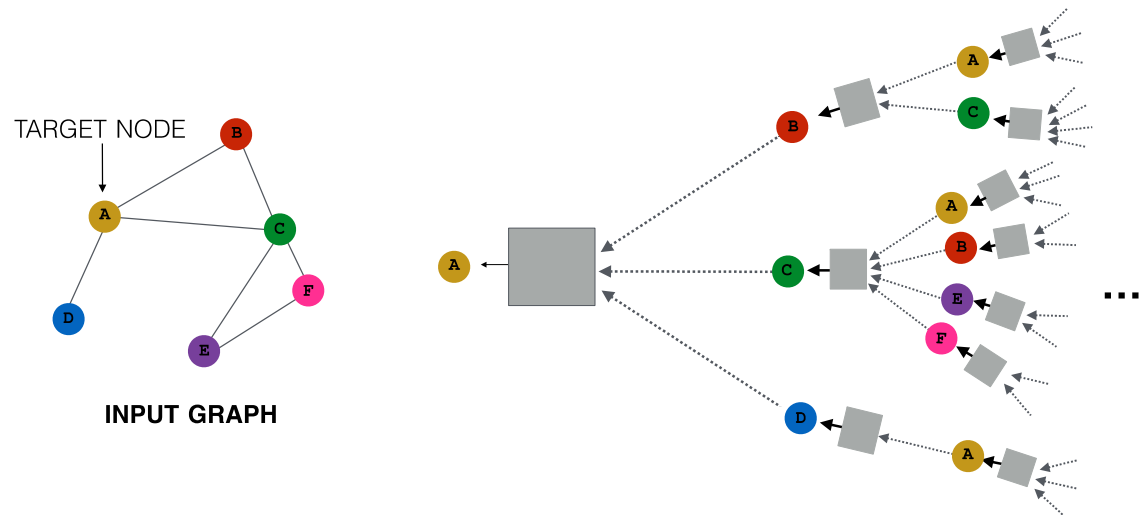
Gated Graph Neural Networks

$$\mathbf{m}_v^k = \mathbf{W} \sum_{u \in N(v)} \mathbf{h}_u^{k-1}$$

$$\mathbf{h}_v^k = \text{GRU}(\mathbf{h}_v^{k-1}, \mathbf{m}_v^k)$$



Gated Graph Neural Networks



- Can handle models with >20 layers.
- Most real-world networks have small diameters (e.g., less than 7).
- Allows for complex information about global graph structure to be propagated to all nodes.

GAT (Graph Attention Networks)

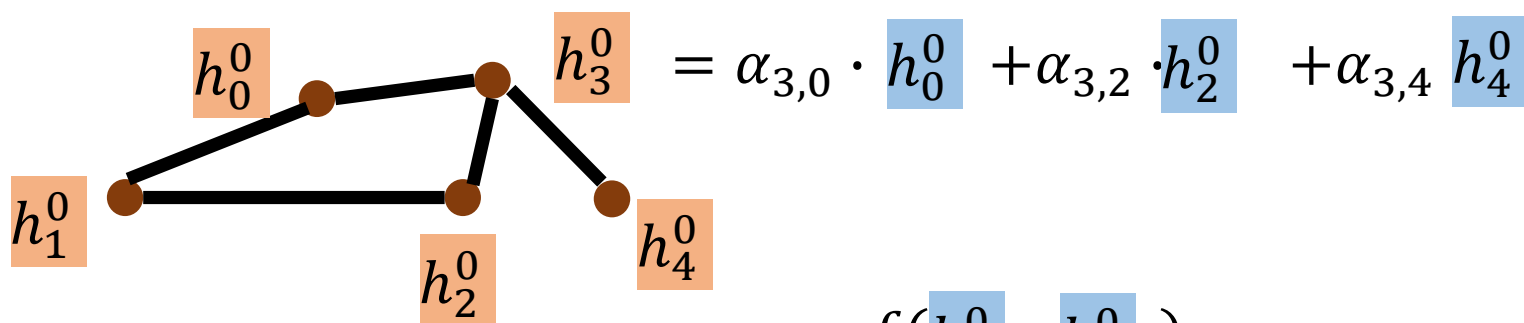
- Input: node features $\mathbf{h} = \{\vec{h}_1, \vec{h}_2, \dots, \vec{h}_N\}, \vec{h}_i \in \mathbb{R}^F$,
- Calculate energy: $e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j)$
- Attention score (over the neighbors)

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k \in \mathcal{N}_i} \exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_k]\right)\right)}$$

<https://arxiv.org/pdf/1710.10903.pdf>

Published as a conference paper at ICLR 2018

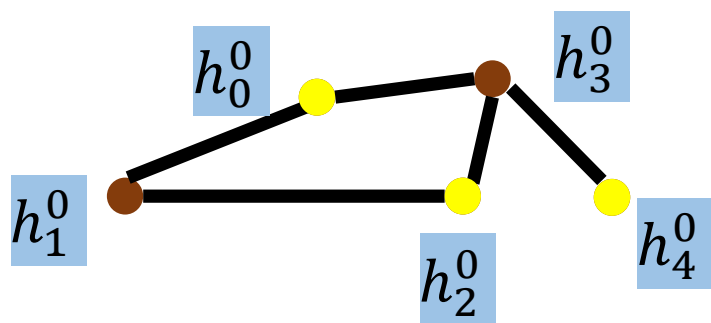
GAT (Graph Attention Networks)



$$f(h_3^0, h_0^0) = e_{3,0}$$

$$f(h_3^0, h_2^0) = e_{3,2}$$

$$f(h_3^0, h_4^0) = e_{3,4}$$



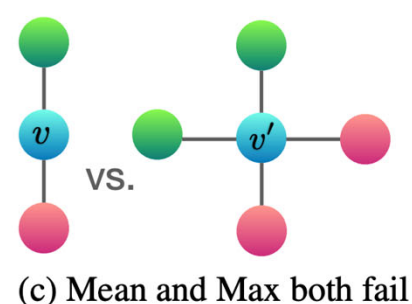
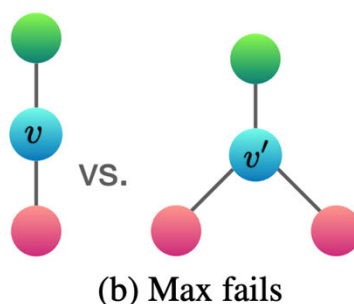
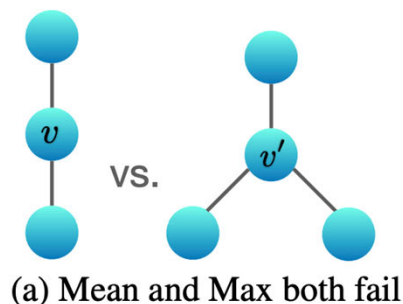
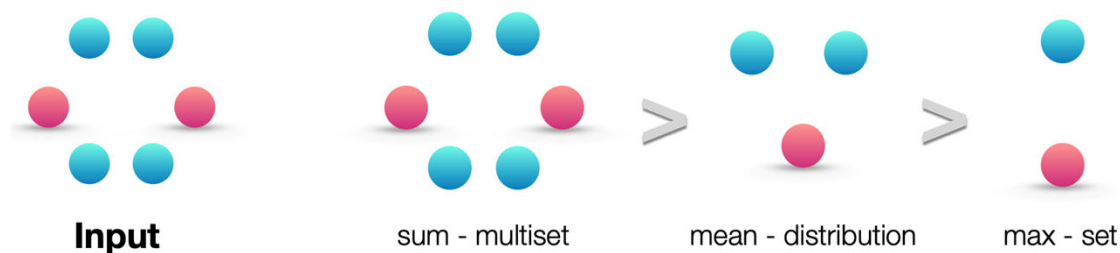
GIN (Graph Isomorphism Network)

- A GNN can be at most as powerful as WL isomorphic test
- Theoretical proofs were provided

GIN (Graph Isomorphism Network)

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)}\right) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right)$$

- ✓ Sum instead of mean or max
- ✓ MLP instead of 1-layer



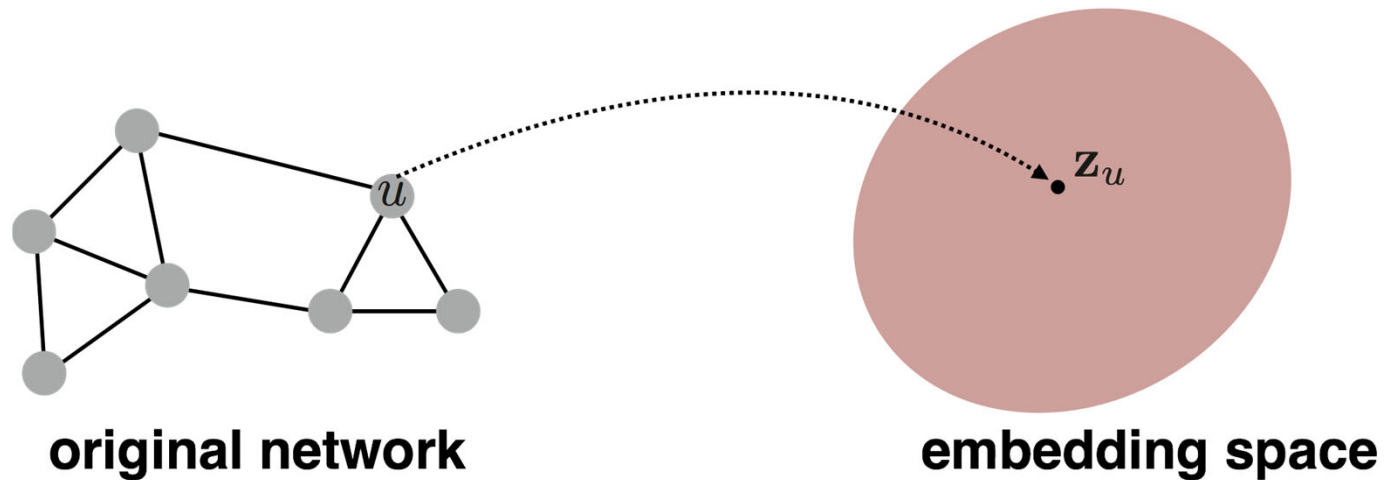
Subgraph Embeddings

Based on material from:

- Duvenaud et al. 2016. [Convolutional Networks on Graphs for Learning Molecular Fingerprints](#). *ICML*.
- Li et al. 2016. [Gated Graph Sequence Neural Networks](#). *ICLR*.

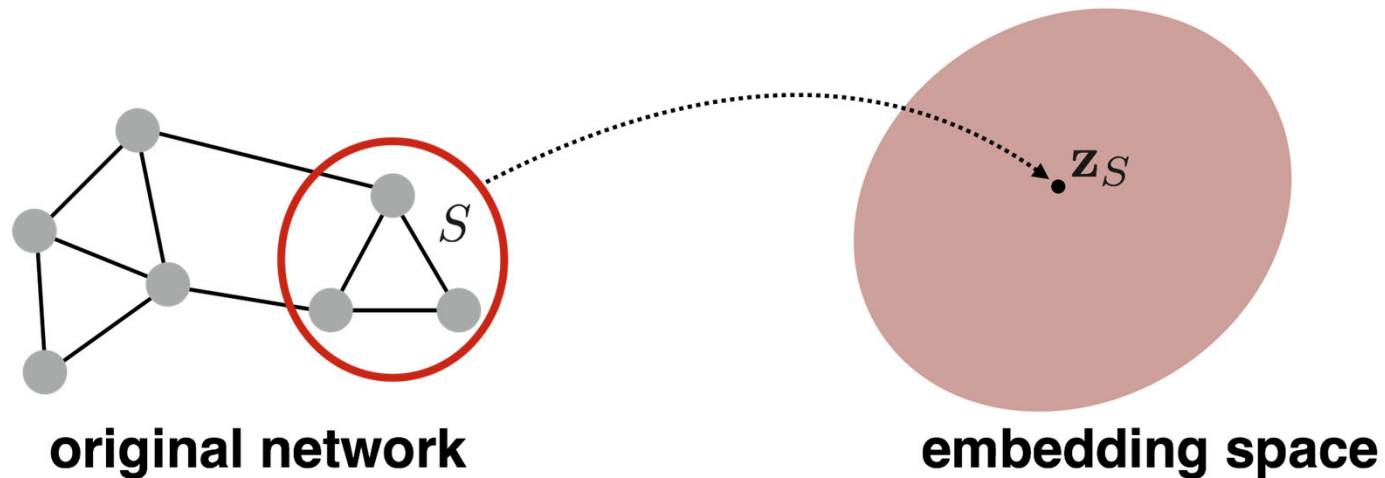
(Sub)graph Embeddings

- So far we have focused on node-level embeddings...



(Sub)graph Embeddings

- But what about **subgraph** embeddings?



Approach 1

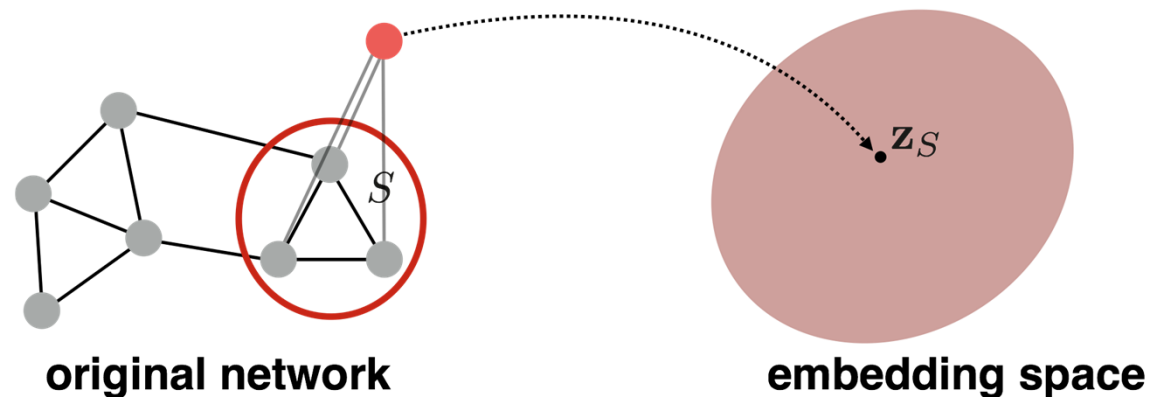
- **Simple idea:** Just sum (or average) the node embeddings in the (sub)graph

$$\mathbf{z}_S = \sum_{v \in S} \mathbf{z}_v$$

- Used by [Duvenaud et al., 2016](#) to classify molecules based on their graph structure.

Approach 2

- **Idea:** Introduce a “**virtual node**” to represent the subgraph and run a standard graph neural network.



- Proposed by [Li et al., 2016](#) as a general technique for subgraph embedding.

Part 3: Resources

Tools

- PyG: the ultimate library for Graph Neural Network
 - <https://www.pyg.org/>
 - https://github.com/pyg-team/pytorch_geometric
- Docs: <https://pytorch-geometric.readthedocs.io/en/latest/>



PyG

Tools

- Deep Graph Library (DGL): Easy deep learning for graphs
 - <https://www.dgl.ai/>
 - <https://github.com/dmlc/dgl>
- Tutorial: <https://docs.dgl.ai/tutorials/blitz/index.html>



Examples

- Unsupervised Neighbour Prediction by GraphSAGE
 - https://github.com/pyg-team/pytorch_geometric/blob/master/examples/graph_sage_unsup.py

Examples

- Link Prediction by GraphSAGE
 - [https://github.com/dglai/KDD20-Hands-on-Tutorial/blob/master/3-basics/3 link predict.ipynb](https://github.com/dglai/KDD20-Hands-on-Tutorial/blob/master/3-basics/3_link_predict.ipynb)